

HospitalOS

Hospital Operating System for Africa

Production Readiness Plan

Engineering reference · Version 1.0 · July 2026 · AgoroX Africa

1. Who this document is for

This is for whoever takes HospitalOS from a working preview to a production deployment — AgoroX engineering, or a contracted team. It covers exactly what each of the five known limitations requires, in what order, and what does *not* need to change.

The short version.

None of the five items below require redesigning HospitalOS. Every service function in the codebase is already written as an async call shaped like a real network request — that was deliberate, from the first module built. Production work happens inside those functions, not in the 67 screens that call them.

2. The five limitations, at a glance

#	Limitation	Current state	Target
1	Data persistence	In-memory, resets on reload	Cloudflare D1
2	Sign-in	Client-side credential check	Server-verified session
3	Audit trail	Tamper-evident in-browser	Server-side append-only store
4	Instruments gateway	Simulated message flow	Real MLLP/DICOM listener
5	Document upload	Session-only blob URLs	Cloudflare R2

Recommended order: 1 and 2 first (nothing else matters if data does not survive and sign-in is not real), then 3 (depends on 1), then 5 (independent, low risk, good early win), then 4 (largest, most independent of the app itself — can run in parallel with 1–3 once scoped).

3. 1. Data persistence — Cloudflare D1

3.1 What exists today

Every module's service file (`patientService.js`, `labService.js`, and so on) holds its data in a JavaScript array or Map at module scope, with async functions (`listX`, `createX`, `updateX`) that already await a small artificial delay to mimic network latency. That async shape is the seam.

3.2 The work

- 1 Design the D1 schema. Each service's in-memory record shape is close to the target table schema already — patients, orders, studies, etc. map close to 1:1.
- 2 Stand up a Worker exposing one API route group per module (or a single generic query/mutation route backed by D1 prepared statements, if preferred).
- 3 Replace each service function's body with a `fetch()` call to the Worker, keeping the exact same function signature — no screen changes.
- 4 Migrate module by module, testing each in isolation. Order by dependency: patients and auth first (everything references a patient), then lab/pharmacy/radiology (reference patients), then billing (reads from all of them), then everything else.



- 5 Multi-tenancy: every table needs a `tenant_id` column and every query needs to scope by it — this is the one piece with no equivalent in the current in-memory build, since the preview only ever runs as a single hospital.

Effort signal.

This is the largest single item. Budget it as the primary body of work; items 3 and 5 both depend on this being in place first.

4. 2. Sign-in — a server-verified session

Credentials currently live in a JavaScript array checked in the browser. The fix is standard, not novel:

- 1 Move the user table into D1, passwords hashed (bcrypt or argon2, never plaintext).
- 2 A Worker sign-in route verifies the password hash and issues a signed session token (JWT, or a D1-backed session row with a secure cookie — either is fine).
- 3 `AuthContext.jsx`'s `signIn()` calls this route instead of checking the in-memory array; the rest of the app (`can()`, `may()`, `isPlatformAdmin`) is unchanged — it already consumes whatever `AuthContext` gives it.
- 4 Every Worker route that mutates data re-verifies the session token server-side before acting — the same principle referenced in the LabOS platform admin guide: a disabled button in the UI is never the real security boundary, the server check is.

Do this alongside item 1.

Sign-in and data persistence are two ends of the same Worker — build them together.

5. 3. Audit trail — server-side append-only

The hash-chain design (`src/lib/audit.js`) does not change. What changes is where entries live.

- 1 An `audit_log` table in D1: append-only by convention — the Worker route exposes INSERT only, never UPDATE or DELETE, for this table specifically.
- 2 `record()` posts to this route instead of pushing into the in-memory array. Same function signature, same call sites, zero screen changes.
- 3 `verifyChain()` becomes a Worker route that re-reads the table and recomputes the chain server-side — the same algorithm, just reading from D1 instead of memory.

Why this is worth doing properly.

An append-only D1 table with no DELETE route is a real control — an attacker would need database-level access, not just browser dev tools, to break it. That is the entire point of migrating this one.

6. 4. Instruments gateway — real device listening

This is the item most independent of the rest of the app, and the one requiring an architecture decision, not just a migration.

6.1 The core problem

MLLP (HL7), DICOM, and most hospital printers are LAN-local protocols — the analyzers and imaging consoles sit on the hospital's internal network, not the public internet. A Cloudflare Worker cannot open a socket to a machine on a hospital's LAN. This needs something running inside the hospital's network.

6.2 The pattern

A small on-premise companion service — a lightweight desktop or server application installed at the hospital, on the same network as the analyzers and imaging consoles. It listens for real MLLP/DICOM messages locally and forwards them to the HospitalOS API over an authenticated HTTPS connection.



- 1 Build the companion service (Node or Go is a reasonable choice) with an MLLP listener and a DICOM SCP, both already well-supported by existing open libraries.
- 2 It authenticates to the HospitalOS Worker with a per-hospital API key, generated from Administration when a hospital enables the gateway.
- 3 On receiving a real HL7 ORU^R01 or DICOM C-STORE, it POSTs to the same `postResultMessage / receiveDicomStudy` routes the in-app simulation already calls — the mapping and validation logic in `instrumentsService.js` does not need to be rewritten, only re-hosted behind a real network listener instead of a UI button.
- 4 Printers are simpler — IPP and raw-socket printing can often be reached directly from a Worker if the printer has a public-reachable address, but for hospital LANs the same companion service is the more reliable path.

Scope note.

This is genuinely independent engineering — it does not block or get blocked by items 1, 2, 3, or 5, and can be scoped and built in parallel.

7. 5. Document upload — Cloudflare R2

The most contained item.

- 1 Create an R2 bucket per tenant, or a shared bucket with a tenant-prefixed key.
- 2 `uploadDocument()` posts the file to a Worker route that streams it into R2 and stores the resulting object key (not the file itself) in D1, alongside the existing metadata (category, uploader, timestamp).
- 3 Replace `URL.createObjectURL(file)` with a signed, time-limited R2 download URL fetched from the Worker — the download link in the UI is unchanged in shape, only in what it points to.

Do this early.

Low risk, clearly bounded, and a good confidence-building first production migration once item 1's Worker exists.

8. What does not need to change

- Every screen's component code — none of the 67 routes need rewriting; they call service functions with fixed signatures that stay fixed.
- The RBAC design (`src/lib/rbac.js`) — `area/action` permission strings move to D1 unchanged, the `canAccessArea()/canDo()` logic is unchanged.
- The audit hash-chain algorithm — only its storage location changes.
- The pricing engine's `priceFor()` seam — already the single source every billing calculation reads through; only its backing store moves to D1.
- The FHIR resource mapping logic — already correct; only the Bundle's transport changes from file-download to a live REST endpoint.

9. Suggested sequencing

- Phase 1: D1 schema + Worker API + sign-in (items 1 & 2 together)
- Phase 2: Migrate audit trail onto the same Worker (item 3)
- Phase 3: Document storage onto R2 (item 5) — can run in parallel with Phase 2
- Phase 4: Instruments gateway companion service (item 4) — independent track, start scoping as early as resourcing allows

Questions about this plan?



support@agorox.africa